0053. Maximum Subarray

Medium · Greedy

leetcode.com/problems/maximum-subarray

Problem

- Return the largest sum of a contiguous non-empty subarray.

Intuition

- Kadane's algorithm: a running sum that has gone negative can only hurt whatever comes next, so drop it and start fresh.

Brute Force

- All subarrays: O(n^2) with running sums.

Optimal Approach

- Cur = 0, best = -infinity.
- For x: cur = max(x, cur + x); best = max(best, cur).

Worked Example

```
[-2,1,-3,4,-1,2,1,-5,4].

cur : -2  1 -2  4  3  5  6  1  5
best: -2  1  1  4  4  5  6  6  6
answer 6 ([4,-1,2,1])
```

Java Solution

```java
public class MaximumSubarray {
    public int maxSubArray(int[] nums) {
        int cur = 0, best = Integer.MIN_VALUE;
        for (int x : nums) {
            cur = Math.max(x, cur + x);
            best = Math.max(best, cur);
        }
        return best;
    }
}
```

Complexity

- Time: O(n). Space: O(1).

Pitfalls

- All-negative input: the answer is the largest element, so do not start best at 0.